A BAG REDUCTION FOR THE RANDIĆ–RADIUS PROBLEM: GRAPHS WITH SMALL TWO-EDGE-CONNECTED COMPONENTS, AND EXHAUSTIVE VERIFICATION THROUGH ELEVEN VERTICES

ANONYMOUS

Abstract. For a graph G let $R(G) = \sum_{uv\in E(G)} (d_u d_v)^{-1/2}$ be the Randić index and $R'(G) = \sum_{uv\in E(G)} 1/\max(d_u, d_v)$ its modified version; always $R \geq R'$. Fajtlowicz's conjecture (1988) asserts $R(G) \geq \mathrm{rad}(G) - 1$ for every connected graph, and Caporossi and Hansen (2000) conjectured $R(G) \geq \mathrm{rad}(G)$ for every connected graph other than a path on an even number of vertices. *Both conjectures remain open; this paper does not prove either of them.* We study the stronger inequality $R'(G) \geq \mathrm{rad}(G) - \frac{1}{2}$, proved for chemical graphs by Cygan, Pilipczuk and Škrekovski. Our results are the following. (i) In every class of graphs closed under connected subgraphs and under replacing a shallow branch behind a bridge by a path, a counterexample with the fewest vertices (and then edges) consists of one bridgeless core with pendant paths, and the associated *bag* violates the inequality $R'(H) \geq \mathrm{rad}(C(H)) + 1$ of Cygan, Pilipczuk and Škrekovski. (ii) An exhaustive computation in exact arithmetic shows that this bag inequality holds for every bag whose core has at most 11 vertices. (iii) Consequently $R'(G) \geq \mathrm{rad}(G) - \frac{1}{2}$, and hence Fajtlowicz's bound, holds for every connected graph each of whose two-edge-connected components has at most 11 vertices, or cyclomatic number at most 5, or only vertices of degree at most four. (iv) We prove an exact conservation identity for supportive edges and derive from it an antichain inequality. (v) We verify (in exact arithmetic for R', and with rigorous floating-point error control for R) that all 1,018,690,328 connected graphs with $2 \leq n \leq 11$ vertices satisfy $R' \geq \mathrm{rad} - \frac{1}{2}$, that those with at most one vertex of degree one satisfy $R' \geq \mathrm{rad}$, and that the Caporossi–Hansen inequality holds for all of them, with equality exactly for $K_2, C_4, C_6, C_8, C_{10}$. All programs are reproduced in the appendix.

1. Introduction

All graphs are finite and simple. For a connected graph G and a vertex v we write d_v for the degree, $\mathrm{ecc}(v) = \max_w d(v, w)$ for the eccentricity, $\mathrm{rad}(G) = \min_v \mathrm{ecc}(v)$, $\mathrm{diam}(G) = \max_v \mathrm{ecc}(v)$ and $\mu(G) = |E(G)| - |V(G)| + 1$ for the cyclomatic number. The Randić index [13] and the modified Randić index of Dvořák, Lidický and Škrekovski [8] are

$$R(G) = \sum_{uv\in E(G)} \frac{1}{\sqrt{d_u d_v}}, \qquad R'(G) = \sum_{uv\in E(G)} \frac{1}{\max(d_u, d_v)}.$$

Since $\sqrt{d_u d_v} \leq \max(d_u, d_v)$, we have $R(G) \geq R'(G)$, with equality exactly when every component is regular.

Conjecture 1.1 (Fajtlowicz [9])**.** *Every connected graph G satisfies $R(G) \geq \mathrm{rad}(G) - 1$.*

Conjecture 1.2 (Caporossi and Hansen [2])**.** *Every connected graph G that is not a path on an even number $n \geq 4$ of vertices satisfies $R(G) \geq \mathrm{rad}(G)$.*

The even paths must be excluded, since $R(P_{2k}) = k - \frac{3}{2} + \sqrt{2} < k = \mathrm{rad}(P_{2k})$ for $k \geq 2$.

Date: October 2026.

Known results. For trees, Caporossi and Hansen proved $R(T) \geq \mathrm{rad}(T) + \sqrt{2} - \frac{3}{2}$ [2], and Deng, Tang and Zhang proved $R(T) > \mathrm{rad}(T) + \frac{1}{15}$ for every tree other than an even path [5], which settles Conjecture 1.2 for trees. Dvořák, Lidický and Škrekovski [8] proved $R'(G) \geq \mathrm{diam}(G)/2$ for every connected graph, which gives $R(G) \geq \mathrm{rad}(G)/2$; Yang and Lu [14] proved the sharp bound $R(G) - \mathrm{diam}(G)/2 \geq \sqrt{2} - 1$ for $n \geq 3$. Cygan, Pilipczuk and Škrekovski [4] proved $R'(G) \geq \mathrm{rad}(G) - \frac{1}{2}$ for every connected graph of maximum degree at most four, and Conjecture 1.2 for such graphs. Deng, Tang and Zhang [5] proved $R(G) \geq \mathrm{rad}(G) - \frac{31}{105}(\mu(G) - 1)$ when $\mu(G) \geq 1$; this settles Conjecture 1.1 for $\mu \leq 4$. Doig [6, 7] verified Conjecture 1.2 for cacti. Further special classes were treated by You and Liu [15] and others; see [8]. An unrefereed manuscript in a public corpus [3] develops a structure theory for minimal counterexamples to Conjecture 1.2 (supportive edges, radius-stable peeling, rooted Yang–Lu replacement) and reports a computer verification for $\mu \leq 8$; Section 3 adapts that approach to R'. As far as we know, Conjectures 1.1 and 1.2 are open in general.

The inequality studied here. We work with the modified index and the inequality

$$R'(G) \;\geq\; \mathrm{rad}(G) - \tfrac{1}{2} \qquad \text{for every connected graph } G. \tag{A}$$

Since $R \geq R'$, inequality (A) implies $R(G) \geq \mathrm{rad}(G) - \frac{1}{2}$ and hence Conjecture 1.1. It is sharp: $R'(P_{2k}) = k - \frac{1}{2}$. Cygan, Pilipczuk and Škrekovski proved (A) for maximum degree at most four; the degree bound enters their proof only through an inequality for *bags* (Definition 3.3), and they remarked that this inequality for all bags would imply Conjecture 1.1 [4]. Inequality (A) is open.

Results. A class $\mathcal{C}$ of connected graphs is *admissible* if it is closed under taking connected subgraphs and under the pendant-path replacement of Lemma 2.7. The class of all connected graphs is admissible.

Theorem A (Theorems 3.2 and 3.4)**.** *Let $\mathcal{C}$ be admissible and let $G \in \mathcal{C}$ violate* (A)*, with $|V(G)|$ minimal and then $|E(G)|$ minimal. Then the 2-core K of G is bridgeless, G is K together with pendant paths, every vertex of K carries at most one of them, at most one has length ≥ 2, and $|V(K)| \leq 2\mu(G) + 1$. The graph H obtained by shortening every pendant path to a single edge is a bag with core K, and $R'(H) < \mathrm{rad}(K) + 1$.*

Theorem B (Theorem 4.1)**.** *Every bag H whose core has at most 11 vertices satisfies $R'(H) \geq \mathrm{rad}(C(H)) + 1$. Equality holds only for $K_{2,3}$ with one pendant vertex attached to each of its five vertices.*

Theorem C (Theorem 4.3)**.** *Let G be a connected graph such that every two-edge-connected component B of G satisfies $|V(B)| \leq 11$, or $\mu(B) \leq 5$, or $d_G(v) \leq 4$ for all $v \in V(B)$. Then $R'(G) \geq \mathrm{rad}(G) - \frac{1}{2}$. In particular $R(G) \geq \mathrm{rad}(G) - \frac{1}{2}$, so Conjecture 1.1 holds for G.*

The class in Theorem C contains, for example, all graphs obtained by joining arbitrarily many graphs of order at most 11 by bridges, and attaching arbitrary trees; such graphs can have unbounded cyclomatic number and unbounded maximum degree. For cyclomatic number at most four and for chemical graphs the conclusion for R was known [5, 4]; the unrefereed manuscript [3] reports Conjecture 1.2 for cyclomatic number at most eight.

Theorem D (Theorem 5.1 and Theorem 5.4)**.** *Let Q be a set of vertices all of whose outside neighbours have smaller degree. Then the total "injection" $\sum_{v \in Q} p_v\, b_v$ through the edges leaving Q equals the total slack of the strict edges inside Q plus an explicit nonnegative term carried by equal-degree edges. If the strict edges inside Q are supportive, every antichain of the degree order in Q has weight at most the injection below it.*

Theorem E (Theorem 6.1)**.** *Every connected graph on at most* 11 *vertices satisfies* (A)*. Every such graph with at most one vertex of degree one satisfies* $R'(G) \geq \mathrm{rad}(G)$*. Every such graph that is not an even path on at least four vertices satisfies* $R(G) \geq \mathrm{rad}(G)$*, with equality exactly for* K_2*,* C_4*,* C_6*,* C_8*,* C_{10}*.*

Relation to earlier work. Theorem A combines three ingredients: the bag concept and the inequality (B) of Cygan, Pilipczuk and Škrekovski [4]; the reduction tools for minimal counterexamples (supportive edges, peeling, rooted replacement) that the manuscript [3] develops for R and Conjecture 1.2; and the bound of Theorem 2.5. Working with R' instead of R makes every identity rational, so that all computations are exact, and the reduction ends in an inequality that involves only the core and its pendant vertices. This is what allows a certification core by core, independently of the cyclomatic number of the whole graph (Theorem C), whereas [3] bounds the order of the core through the cyclomatic number. Theorem D and the exhaustive verification for $n = 11$ are new as far as we know.

What is not proved. We do not prove Conjecture 1.1, Conjecture 1.2, or inequality (A) in general. By Theorem A, inequality (A) for all graphs would follow from the bag inequality for all bags (Conjecture 7.1); we verify that inequality only for cores of order at most 11. Section 7 lists the remaining gap and further conjectures, together with the evidence for them.

2. Preliminaries

Let G be a graph. For a vertex v with $d_v \geq 2$ put

$$\lambda_v = |\{w \in N(v) : d_w < d_v\}|, \quad h_v = d_v - \lambda_v, \quad g_v = d_v(d_v - 1), \quad a_v = \frac{\lambda_v}{g_v}, \quad b_v = \frac{h_v}{g_v}.$$

Thus λ_v counts the *lower* neighbours of v, h_v counts the neighbours of degree at least d_v, and $a_v + b_v = \frac{1}{d_v - 1}$. A vertex with $\lambda_v = 0$ is *locally minimal*. A vertex of degree one is a *leaf*, and its neighbour is its *support*. An edge e is *supportive* if $R'(G - e) > R'(G)$.

Lemma 2.1 (Edge deletion)**.** *Let* $uv \in E(G)$ *with* $d_u, d_v \geq 2$*, and let* β_u *be the number of neighbours* $w \neq v$ *of* u *with* $d_w < d_u$ *(and symmetrically* β_v*). Then*

$$R'(G - uv) - R'(G) = -\frac{1}{\max(d_u, d_v)} + \frac{\beta_u}{g_u} + \frac{\beta_v}{g_v}.$$

In particular:

(i) if $d_u < d_v$*, then* $R'(G - uv) - R'(G) = a_u - b_v$*, so* uv *is supportive if and only if* $a_u > b_v$*;*
(ii) if $d_u = d_v = k$*, then* $R'(G - uv) - R'(G) = \frac{\lambda_u + \lambda_v}{k(k-1)} - \frac{1}{k}$*, so* uv *is supportive if and only if* $\lambda_u + \lambda_v \geq k$*.*

Proof. Only the terms of edges at u or v change. The term of uv disappears. For $w \in N(u) \setminus \{v\}$ the term $1/\max(d_u, d_w)$ becomes $1/\max(d_u - 1, d_w)$: it is unchanged if $d_w \geq d_u$, and it increases from $1/d_u$ to $1/(d_u - 1)$, that is by $1/g_u$, if $d_w < d_u$. The same holds at v. This proves the formula. If $d_u < d_v$ then $\beta_u = \lambda_u$ and $\beta_v = \lambda_v - 1$, and $-\frac{1}{d_v} + \frac{\lambda_v - 1}{g_v} = -\frac{d_v - \lambda_v}{g_v} = -b_v$. If $d_u = d_v = k$ then $\beta_u = \lambda_u$, $\beta_v = \lambda_v$, and the expression is positive iff $\lambda_u + \lambda_v > k - 1$. □

Lemma 2.2 (Locally minimal vertices)**.** *If* $d_u \geq 2$ *and* $\lambda_u = 0$*, then no edge at* u *is supportive.*

Proof. Let uv be an edge. As u has no lower neighbour, $d_v \geq d_u$. If $d_v > d_u$, Lemma 2.1(i) gives $R'(G - uv) - R'(G) = a_u - b_v = -b_v \leq 0$. If $d_v = d_u = k$, then $\lambda_v \leq k - 1$ because u is not lower than v, and Lemma 2.1(ii) gives $\frac{\lambda_v}{k(k-1)} - \frac{1}{k} \leq 0$. □

Lemma 2.3 (Deleting leaves)**.** *Let L be a set of leaves of G with pairwise distinct supports, let S be the set of these supports, and assume $d_s \geq 2$ for $s \in S$. Then, with all quantities computed in G,*

$$R'(G) - R'(G - L) = \sum_{s \in S} b_s - \sum_{\substack{st \in E(G),\ s,t \in S \\ d_s = d_t}} \frac{1}{g_s} \geq 0.$$

In particular $R'(G) - R'(G - \ell) = b_s \geq 0$ for a single leaf ℓ with support s.

Proof. Deleting L lowers the degree of each $s \in S$ by one and changes no other degree. The leaf edge at s contributes $1/d_s$ and disappears. Consider an edge sw, $s \in S$, $w \notin L$. If $w \notin S$, its term increases by $1/g_s$ when $d_w < d_s$ and is unchanged otherwise. If $w \in S$, its term $1/\max(d_s, d_w)$ becomes $1/\max(d_s - 1, d_w - 1)$; it increases by $1/g_s$ when $d_w < d_s$, by $1/g_w$ when $d_w > d_s$, and by $1/g_s$ when $d_w = d_s$. Collecting, the total increase equals $\sum_{s \in S}(\lambda_s - 1)/g_s$ plus one term $1/g_s$ for each equal-degree edge inside S (the leaf at s is one of the λ_s lower neighbours). Since $\frac{1}{d_s} - \frac{\lambda_s - 1}{g_s} = \frac{h_s}{g_s} = b_s$, the formula follows. For nonnegativity delete the leaves one at a time; by the case $|L| = 1$ each step lowers R' by some $b \geq 0$. □

Lemma 2.4 (Peeling)**.** *Let $S \subset V(G)$ be such that $G - S$ is connected and every vertex of S has a neighbour outside S. Then* $\mathrm{rad}(G - S) \geq \mathrm{rad}(G) - 1$.

Proof. Let c be a centre of $G - S$. Every vertex outside S is within distance $\mathrm{rad}(G - S)$ of c in $G - S$, hence in G, and every vertex of S is adjacent to such a vertex. So $\mathrm{ecc}_G(c) \leq \mathrm{rad}(G - S) + 1$. □

Theorem 2.5 (Dvořák, Lidický and Škrekovski [8])**.** *Every connected graph G satisfies $R'(G) \geq \mathrm{diam}(G)/2$.*

For a connected graph H and $y \in V(H)$ let $R'_y(H)$ denote the value of $R'(H)$ computed with the degree of y replaced by $d_H(y) + 1$ (as if y had one more neighbour outside H).

Lemma 2.6 (Rooted form of Theorem 2.5)**.** *If H is connected and $y \in V(H)$ has $\mathrm{ecc}_H(y) = e$, then $R'_y(H) \geq \frac{e+1}{2} - \frac{1}{d_H(y)+1}$.*

Proof. Add a new vertex z adjacent only to y. The new graph H^+ satisfies $R'(H^+) = R'_y(H) + \frac{1}{d_H(y)+1}$ and $\mathrm{diam}(H^+) \geq \mathrm{ecc}_{H^+}(z) = e + 1$. Apply Theorem 2.5 to H^+. □

Lemma 2.7 (Pendant-path replacement)**.** *Let xy be a bridge of a connected graph G, let $G_x \ni x$ and $G_y \ni y$ be the components of $G - xy$, and put $e_x = \mathrm{ecc}_{G_x}(x)$, $e_y = \mathrm{ecc}_{G_y}(y)$. Assume $e_y \leq e_x$. Let G' be obtained from G_x by attaching at x a new path with $e_y + 1$ edges. Then $R'(G') \leq R'(G)$ and $\mathrm{rad}(G') \geq \mathrm{rad}(G)$. Moreover $|V(G')| \leq |V(G)|$, with equality if and only if G_y is a path having y as an end vertex.*

Proof. If $G_y = \{y\}$ there is nothing to prove, so let $e_y \geq 1$. The vertex x has the same degree d_x in G and G', so the edges of G_x contribute the same amount to $R'(G)$ and $R'(G')$. Put $d_y = d_G(y) \geq 2$. The remaining contributions are $\frac{1}{\max(d_x, d_y)} + R'_y(G_y)$ in G and $\frac{1}{\max(d_x, 2)} + \frac{e_y}{2}$ in G' (all vertices of the new path except its end have degree two). By Lemma 2.6,

$$R'(G) - R'(G') \ \geq \ \frac{1}{2} + \frac{1}{\max(d_x, d_y)} - \frac{1}{d_y} - \frac{1}{\max(d_x, 2)}.$$

If $d_x \leq d_y$ the right side is $\frac{1}{2} - \frac{1}{\max(d_x, 2)} \geq 0$; if $d_x > d_y \geq 2$ it is $\frac{1}{2} - \frac{1}{d_y} \geq 0$.

For the radius, a vertex $v \in V(G_x)$ has $\mathrm{ecc}_{G'}(v) = \max(\mathrm{ecc}_{G_x}(v), d(v, x) + 1 + e_y) = \mathrm{ecc}_G(v)$. A vertex of the new path at distance $j \geq 1$ from x has eccentricity at least $j + e_x \geq 1 + e_x$ in G', while $\mathrm{rad}(G) \leq \mathrm{ecc}_G(x) = \max(e_x, 1 + e_y) \leq 1 + e_x$. Hence $\mathrm{rad}(G') \geq \mathrm{rad}(G)$. Finally $|V(G_y)| \geq e_y + 1$, with equality exactly when G_y is a path starting at y. □

3. Minimal counterexamples

Definition 3.1. A class $\mathcal{C}$ of connected graphs is *admissible* if (a) every connected subgraph of a member of $\mathcal{C}$ is in $\mathcal{C}$, and (b) whenever $G \in \mathcal{C}$ and xy, G_x, G_y, G' are as in Lemma 2.7, also $G' \in \mathcal{C}$.

Theorem 3.2 (Structure)**.** *Let $\mathcal{C}$ be admissible and let $G \in \mathcal{C}$ violate* (A)*, with $|V(G)|$ minimal and, subject to this, $|E(G)|$ minimal among such graphs in $\mathcal{C}$. Put $r = \mathrm{rad}(G)$ and let K be the 2-core of G. Then:*

(a) G is not a tree and $\mathrm{diam}(G) \le 2r - 2$;
(b) every edge of G lying on a cycle is supportive;
(c) K is connected and bridgeless, and $E(G) \setminus E(K)$ is the set of bridges of G;
(d) every component of $G - E(K)$ with at least two vertices is a path that meets K in exactly one vertex, which is one of its ends (a pendant path*); every vertex of K is an end of at most one pendant path; there is at least one pendant path, and at most one of them has length at least two;*
(e) every vertex of K has degree at least three in G and is not locally minimal; every vertex whose degree in K is two or three carries a pendant path;
(f) at most three vertices have degree two in K; consequently $|V(K)| \le 2\mu(K)+1 = 2\mu(G)+1$;
(g) for every leaf ℓ we have $\mathrm{rad}(G - \ell) = r - 1$, and for every nonempty set L of leaves $R'(G) - R'(G - L) < 1$.

Proof. Throughout, every graph obtained from G by the operations below lies in $\mathcal{C}$ by admissibility, and if it violates (A) it contradicts the minimality of G.

(a) If $\mathrm{diam}(G) \ge 2r - 1$, Theorem 2.5 gives $R'(G) \ge r - \frac{1}{2}$. A tree has diam $\ge 2\,\mathrm{rad} - 1$.

(b) Let e lie on a cycle. Then $G - e$ is connected, has the same vertices and fewer edges, and $\mathrm{rad}(G - e) \ge r$ because distances do not decrease. If $R'(G - e) \le R'(G)$ then $G - e$ violates (A).

Bridges. Let xy be a bridge with $e_y \le e_x$ (notation of Lemma 2.7). If G_y is not a path starting at y, the graph G' of Lemma 2.7 has fewer vertices, $R'(G') \le R'(G) < r - \frac{1}{2} \le \mathrm{rad}(G') - \frac{1}{2}$, a contradiction. So: *for every bridge, a side of smaller or equal eccentricity is a path starting at the bridge.*

(c) Since G is not a tree, $K \neq \emptyset$; K is connected, all cycles of G lie in K, and the edges outside K are bridges. Suppose an edge xy of K is a bridge of G. On the side G_x, every vertex of K other than x keeps its K-degree (at least two), and x keeps K-degree at least one; if $K \cap G_x$ were a forest, it would be the single vertex x (impossible, as $d_K(x) \ge 2$) or would have two vertices of degree at most one. So G_x contains a cycle, and so does G_y. Then neither side is a path, contradicting the previous paragraph. Hence K is bridgeless and the bridges of G are exactly the edges outside K.

(d), first part. The components of $G - E(K)$ are trees, each meeting K in one vertex. Let wy be an edge with $w \in V(K)$, $y \notin V(K)$. The side G_w contains a cycle, so it is not a path; by the bridge statement applied in both directions we get $e_y < e_w$ and that G_y is a path starting at y. Hence every branch at a vertex of K is a pendant path, and a pendant path with s edges attached at w satisfies $s \le \mathrm{ecc}_{G_w}(w)$.

(g), first part. Let ℓ be a leaf with support s. By Lemma 2.3, $R'(G - \ell) \le R'(G)$. If $\mathrm{rad}(G - \ell) \ge r$, the graph $G - \ell$ violates (A). Hence $\mathrm{rad}(G - \ell) \le r - 1$, and equality holds by Lemma 2.4. Choose c with $\mathrm{ecc}_{G-\ell}(c) \le r - 1$. Since $\mathrm{ecc}_G(c) \ge r$ and deleting a leaf does not change the other distances, $d(c, \ell) = r$, and every other vertex is within distance $r - 1$ of c.

(d), second part. Suppose two leaves ℓ, ℓ' have the same support s, and let c be as above for ℓ. If $c \neq \ell'$, then $d(c, \ell') = d(c, s) + 1 = d(c, \ell) = r$, although ℓ' is a vertex of $G - \ell$; this is impossible. If $c = \ell'$, then every vertex of $G - \ell$ is within distance $r - 1 = d(c, \ell) - 1 = 1$ of the leaf ℓ', so G is the path $\ell s \ell'$, contradicting (a). Hence distinct leaves have distinct supports. Suppose w carries

a leaf ℓ and a pendant path $wp_1 \cdots p_s$ with $s \geq 2$. With c as above for ℓ we have $d(c, w) = r - 1$. If c is not on this path, $d(c, p_s) = r - 1 + s > r - 1$; if $c = p_j$, then $j = r - 1$ and a vertex $z \neq w$ of K satisfies $d(c, z) \geq j + 1 = r$. Both contradict the choice of c. Finally, suppose two pendant paths of length at least two end in leaves q_1, q_2 whose supports t_1, t_2 have degree two. Then $t_1 t_2 \notin E(G)$, and by Lemma 2.3 $R'(G) - R'(G - \{q_1, q_2\}) = b_{t_1} + b_{t_2} = \frac{1}{2} + \frac{1}{2} = 1$. By Lemma 2.4, $\operatorname{rad}(G - \{q_1, q_2\}) \geq r - 1$, so $G - \{q_1, q_2\}$ violates (A). If there were no pendant path, $G = K$ would be bridgeless with minimum degree at least two; a vertex of minimum degree is locally minimal, and its edges, which lie on cycles, would be non-supportive by Lemma 2.2, contradicting (b).

(g), second part. Let L be a nonempty set of leaves. Their supports are distinct and are not leaves, so Lemma 2.4 applies and $\operatorname{rad}(G - L) \geq r - 1$. If $R'(G) - R'(G - L) \geq 1$ then $R'(G - L) < r - \frac{3}{2} \leq \operatorname{rad}(G - L) - \frac{1}{2}$.

(e) Each vertex v of K lies on a cycle, so by (b) and Lemma 2.2 it is not locally minimal. If $d_K(v) = 2$ and v carries no pendant path, then $d_v = 2$ and all neighbours of v have degree at least two, so v is locally minimal; hence every vertex of K-degree two carries a pendant path and has degree three. Then every vertex of K has degree at least three, and a vertex of K-degree three without pendant path would have degree three and only neighbours of degree at least three, i.e. it would be locally minimal.

(f) Let x have K-degree two and carry a pendant path of length one (a leaf). Then $d_x = 3$, $\lambda_x = 1$, $h_x = 2$ and $b_x = \frac{1}{3}$. Two such vertices are not adjacent: an edge between them joins two vertices of degree three with $\lambda_x + \lambda_{x'} = 2 < 3$, which is not supportive by Lemma 2.1(ii). If there were three such vertices, their leaves L would satisfy $R'(G) - R'(G - L) = 3 \cdot \frac{1}{3} = 1$ by Lemma 2.3, contradicting (g). By (d) at most one further vertex of K-degree two carries the long pendant path. Hence at most three vertices have K-degree two, and $2\mu(K) - 2 = \sum_{v \in V(K)} (d_K(v) - 2) \geq |V(K)| - 3$. Pendant paths do not change the cyclomatic number. □

Definition 3.3 (Cygan, Pilipczuk and Škrekovski [4])**.** A connected graph H is a *bag* if (1) deleting its vertices of degree one leaves a bridgeless connected graph $C(H)$, the *core*; (2) every vertex of H has at most one neighbour of degree one; and (3) every edge of $C(H)$ is supportive in H. The *bag inequality* for H is

$$R'(H) \;\geq\; \operatorname{rad}(C(H)) + 1. \tag{B}$$

Cygan, Pilipczuk and Škrekovski proved (B) for bags of maximum degree at most four [4, Proposition 4.3].

Theorem 3.4 (Bag reduction)**.** *Let G and K be as in Theorem 3.2, and let H be obtained from G by deleting, on every pendant path, all vertices except the one adjacent to K. Then H is a bag with core K, and H violates* (B)*: $R'(H) < \operatorname{rad}(K) + 1$.*

Proof. By Theorem 3.2(d),(e), the vertices of K have degree at least three in H, the other vertices of H are leaves, and every vertex of K has at most one leaf neighbour; so (1) and (2) hold with $C(H) = K$. Every vertex of K has the same degree in H as in G, and the first vertex of a pendant path has degree one or two in G and degree one in H, in both cases smaller than the degree of its neighbour in K. By Lemma 2.1, $R'(H - e) - R'(H) = R'(G - e) - R'(G) > 0$ for every $e \in E(K)$ (Theorem 3.2(b)), which is (3).

Suppose $R'(H) \geq \operatorname{rad}(K) + 1$, and let c be a centre of K, $r_K = \operatorname{rad}(K)$. Distances between vertices of K are the same in K and in G. If all pendant paths have length one, then $G = H$ and $\operatorname{ecc}_G(c) \leq r_K + 1$, so $R'(G) \geq r_K + 1 \geq \operatorname{rad}(G)$, contradicting the choice of G. Otherwise there is exactly one pendant path $wp_1 \cdots p_s$ with $s \geq 2$. Its edges $p_i p_{i+1}$ contribute $\frac{1}{2}$ each, and the edge wp_1 contributes $1/d_w$ in both G and H, so $R'(G) = R'(H) + \frac{s-1}{2} \geq r_K + \frac{s+1}{2}$. Let $t = d(c, w) \leq r_K$ and walk from c along a shortest path to w and then along the pendant path; let y be the vertex

at distance $\sigma = \max\bigl(0, \lceil (t+s-r_K-1)/2 \rceil\bigr) \le t+s$ from c on this walk. Every vertex of K and every leaf of G outside the long path is within $d(y,c)+r_K+1 = \sigma+r_K+1$ of y, and every vertex p_i is within $\max(\sigma, t+s-\sigma)$ of y. Hence $\operatorname{ecc}(y) \le \max(\sigma+r_K+1, t+s-\sigma)$. If $\sigma > 0$ this maximum equals $\lceil (t+s+r_K+1)/2 \rceil \le r_K + \lceil (s+1)/2 \rceil$, and if $\sigma = 0$ it equals r_K+1. Therefore $\operatorname{rad}(G) \le r_K + \lceil (s+1)/2 \rceil \le r_K + \frac{s}{2} + 1 \le R'(G) + \frac{1}{2}$, again a contradiction. □

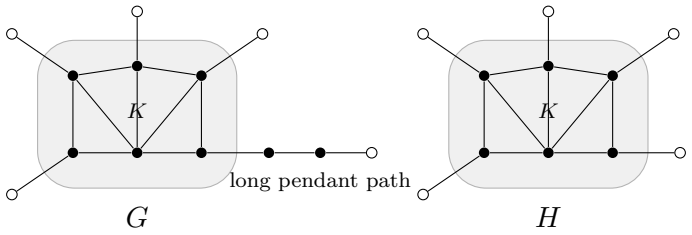


FIGURE 1. Schematic shape of a minimal counterexample G (left) and of its bag H (right) in Theorems 3.2 and 3.4. Shaded: the bridgeless core K; open circles: vertices of degree one. Every vertex of K carries at most one pendant path, at most one pendant path is long, and H keeps only the first vertex of each pendant path.

Corollary 3.5. *If* (B) *holds for every bag, then* (A) *holds for every connected graph, and Conjecture 1.1 follows.*

Proof. Apply Theorems 3.2 and 3.4 to the class of all connected graphs. □

Cygan, Pilipczuk and Škrekovski observed that the general bag inequality would imply Conjecture 1.1. Their reduction is different (it treats all bridgeless components of a minimal counterexample simultaneously); the proof above is self-contained and works inside any admissible class, which is what Section 4 needs.

4. The bag inequality for small cores, and the main theorem

Theorem 4.1 (Computer-assisted)**.** *Every bag H with $|V(C(H))| \le 11$ satisfies* (B)*. Equality holds only when $C(H) = K_{2,3}$ and all five core vertices carry a pendant vertex.*

Method. A bag is determined by its core C and the set $U \subseteq V(C)$ of core vertices carrying a pendant vertex. A core vertex of core-degree at most three, or without a core neighbour of smaller core-degree, must lie in U: otherwise it is locally minimal in H (for core-degree two directly; then for core-degree three, since all core vertices have degree at least three; and in the last case because $d_H(v) = d_C(v) \le d_C(w) \le d_H(w)$ for all neighbours w), and its core edges are not supportive by Lemma 2.2. The program `bags2.c` (Appendix A) reads all connected graphs with minimum degree at least two produced by `geng -c -d2` [10], keeps the bridgeless ones, and for each of them runs through all sets U containing the forced vertices. Condition (3) of Definition 3.3 is tested exactly with Lemma 2.1 in integer arithmetic, and $R'(H)$ is computed exactly as an integer multiple of $1/27720$ ($27720 = \operatorname{lcm}(1,\dots,11)$, and all degrees are at most 11). Table 1 records the outcome. Core order 10 takes about 42 seconds on one core; for core order 11 the input was split into eight parts with `geng`'s `res/mod` option, each taking about 13 to 18 minutes on one core. An independent brute-force script (Appendix A), which uses no forced vertices and no deletion formula but tries every $U \subseteq V(C)$ and recomputes $R'(H-e)$ for every core edge with rational arithmetic, reproduces the numbers of cores and bags, the minimum slack and the minimum drop for every core order up to 7. A variant of `bags2.c` that forces only the vertices of core-degree at

most three (and therefore enumerates more sets U) gives identical results for every core order up to 10. □

$\lvert V(C)\rvert$	bridgeless cores	bags	min slack in (B)	min drop	min $\lvert U\rvert$
3	1	0	–	–	–
4	3	1	5/12	3/4	4
5	11	5	0	1	3
6	60	12	1/3	17/20	4
7	502	80	2/3	59/60	4
8	7 403	504	2/3	13/15	5
9	197 442	5 468	1	1	4
10	9 804 368	82 715	1	19/21	4
11	902 818 087	1 731 733	4/3	1	4

TABLE 1. All bags with small cores. "Slack" is $R'(H) - \mathrm{rad}(C) - 1$; it is never negative, and it is zero only for $K_{2,3}$ with five pendant vertices. "Drop" is $R'(H) - R'(C)$, and $|U|$ is the number of pendant vertices. The numbers of bridgeless cores agree with the numbers of connected bridgeless graphs (OEIS A007146 [12]).

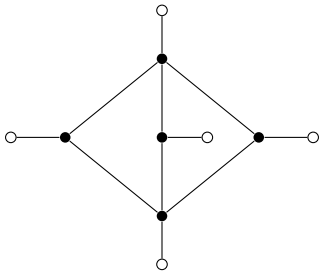


FIGURE 2. The only bag with core order at most 11 attaining equality in (B): $K_{2,3}$ with a pendant vertex at each of its five vertices; $R'(H) = 3 = \mathrm{rad}(K_{2,3}) + 1$.

For the equality case, H consists of $K_{2,3}$ (parts $\{x_1, x_2\}$ and $\{y_1, y_2, y_3\}$) with a pendant vertex at each of the five vertices; then $d_{x_i} = 4$, $d_{y_j} = 3$, $R'(H) = 6 \cdot \frac{1}{4} + 2 \cdot \frac{1}{4} + 3 \cdot \frac{1}{3} = 3 = \mathrm{rad}(K_{2,3}) + 1$, and every core edge $y_j x_i$ has $a_{y_j} - b_{x_i} = \frac{1}{6} - 0 > 0$.

Lemma 4.2. *For integers $k \geq 1$ and $m \geq 0$ let $\mathcal{C}(k, m)$ be the class of connected graphs G such that every two-edge-connected component B of G with at least two vertices satisfies $|V(B)| \leq k$, or $\mu(B) \leq m$, or $d_G(v) \leq 4$ for all $v \in V(B)$. Then $\mathcal{C}(k, m)$ is admissible.*

Proof. Let G' be a connected subgraph of $G \in \mathcal{C}(k, m)$. A two-edge-connected component B' of G' with at least two vertices is a two-edge-connected subgraph of G, so it is contained in a two-edge-connected component B of G. Then $|V(B')| \leq |V(B)|$; $\mu(B') \leq \mu(B)$, because B arises from its connected subgraph B' by adding vertices, each with at least one new edge, and further edges; and $d_{G'}(v) \leq d_G(v)$. So B' satisfies the same alternative as B. The graph G' of Lemma 2.7 consists of the connected subgraph G_x of G and a pendant path at x; its two-edge-connected components with at least two vertices are those of G_x, and all degrees of vertices of G_x in G' are at most their degrees in G. □

Theorem 4.3 (Main theorem)**.** *Every graph $G \in \mathcal{C}(11, 5)$ satisfies* (A)*. That is, if every two-edge-connected component B of a connected graph G has at most* 11 *vertices, or cyclomatic number*

at most 5*, or only vertices of degree at most four in* G*, then* $R'(G) \geq \operatorname{rad}(G) - \frac{1}{2}$*, hence* $R(G) \geq \operatorname{rad}(G) - \frac{1}{2}$ *and Conjecture 1.1 holds for* G*.*

Proof. Suppose not, and let $G \in \mathcal{C}(11, 5)$ be a counterexample with $|V(G)|$ and then $|E(G)|$ minimal. By Lemma 4.2 and Theorem 3.2, the core K is a two-edge-connected component of G. If $\mu(K) \leq 5$, then $|V(K)| \leq 2\mu(K) + 1 \leq 11$ by Theorem 3.2(f). So either $|V(K)| \leq 11$, and Theorem 4.1 gives (B) for the bag H of Theorem 3.4; or all vertices of K have degree at most four in G, hence in H, so H has maximum degree at most four and (B) holds by [4, Proposition 4.3]. Both contradict Theorem 3.4. □

Remark 4.4. Theorem 4.3 uses Theorem 4.1 only for cores that can occur in Theorem 3.4. Using the additional restrictions of Theorem 3.2, for example $R'(H) - R'(K) < 1$ when there is no long pendant path, would shrink the search considerably; we did not need this for core order at most 11.

5. A conservation identity for supportive edges

In this section G is a graph and $Q \subseteq V(G)$ is a nonempty set of vertices of degree at least two such that

(EXT) every neighbour outside Q of a vertex $v \in Q$ has degree smaller than d_v.

For $v \in Q$ let $p_v = |N(v) \setminus Q|$ (the number of *external* neighbours), let $L_Q(v)$ and $U_Q(v)$ be the sets of neighbours of v in Q of smaller and of larger degree, and let e_v be the number of neighbours of v in Q of degree d_v. By (EXT), $\lambda_v = p_v + |L_Q(v)|$ and $h_v = e_v + |U_Q(v)|$. An edge uv of $G[Q]$ with $d_u < d_v$ is *strict*. For $D \subseteq Q$ the *injection* into D is $I(D) = \sum_{v \in D} p_v b_v$.

Theorem 5.1 (Conservation identity)**.**

$$I(Q) = \sum_{\substack{uv \in E(G[Q]) \\ d_u < d_v}} \big(a_u - b_v\big) \; + \; \sum_{v \in Q} e_v \, a_v.$$

Proof. For every v, $\lambda_v b_v = \lambda_v h_v / g_v = h_v a_v$. Summing over Q,

$$\sum_{v \in Q} \big(p_v + |L_Q(v)|\big) b_v = \sum_{v \in Q} \big(e_v + |U_Q(v)|\big) a_v.$$

Now $\sum_v |L_Q(v)| b_v$ is the sum of b_v over all strict edges uv ($d_u < d_v$), and $\sum_v |U_Q(v)| a_v$ is the sum of a_u over the same edges. □

By Lemma 2.1(i), $a_u - b_v = R'(G - uv) - R'(G)$ for a strict edge, so if all strict edges of $G[Q]$ are supportive, the injection $I(Q)$ is a sum of positive "slacks" of strict edges and nonnegative contributions of equal-degree edges. For a bag H with core C and pendant set U, take $Q = V(C)$; then $p_v = 1$ for $v \in U$ and $p_v = 0$ otherwise, and Lemma 2.3 gives

$$R'(H) - R'(C) = I(V(C)) - \sum_{\substack{uv \in E(C),\ u,v \in U \\ d_u = d_v}} \frac{1}{g_u}. \tag{1}$$

We read Theorem 5.1 as a flow statement. Give each strict edge uv ($d_u < d_v$) the flow a_u from u to v, and put $\operatorname{In}(v) = \sum_{u \in L_Q(v)} a_u$ and $\operatorname{Out}(v) = |U_Q(v)| \, a_v$.

Lemma 5.2 (Local conservation)**.** *If every strict edge of* $G[Q]$ *is supportive, then for every* $v \in Q$

$$\operatorname{Out}(v) + e_v a_v \;\leq\; \operatorname{In}(v) + p_v b_v,$$

with strict inequality whenever $L_Q(v) \neq \emptyset$*.*

Proof. $\mathrm{Out}(v) + e_v a_v = h_v a_v = \lambda_v b_v = p_v b_v + |L_Q(v)|\, b_v$, and $b_v < a_u$ for every $u \in L_Q(v)$ by Lemma 2.1(i). □

Call $D \subseteq Q$ a *down-set* if $L_Q(v) \subseteq D$ for every $v \in D$.

Lemma 5.3 (Cut inequality)**.** *If every strict edge of $G[Q]$ is supportive and D is a down-set, then*

$$\sum_{u \in D,\ v \in U_Q(u) \setminus D} a_u \ + \ \sum_{v \in D} e_v a_v \ \le \ I(D).$$

Proof. Sum Lemma 5.2 over D. Because D is a down-set, $\sum_{v \in D} \mathrm{In}(v)$ is the total flow on strict edges inside D, which is also part of $\sum_{v \in D} \mathrm{Out}(v)$; the rest of that sum is the flow leaving D. □

Let $\preceq$ be the reflexive–transitive closure of the relation "$u \in L_Q(v)$". Degrees increase strictly along $\prec$, so $\preceq$ is a partial order on Q. For $A \subseteq Q$ let $\downarrow A = \{v \in Q : v \preceq a \text{ for some } a \in A\}$.

Theorem 5.4 (Antichain inequality)**.** *If every strict edge of $G[Q]$ is supportive, then every antichain A of $(Q, \preceq)$ satisfies*

$$\sum_{a \in A} \lambda_a b_a \ \le \ I(\downarrow A), \qquad \text{and in particular} \qquad \sum_{\substack{a \in A \\ \lambda_a \ge 1,\ h_a \ge 1}} \frac{1}{d_a} \ \le \ I(\downarrow A).$$

Proof. Let $D = \downarrow A \setminus A$. If $v \in D$ and $u \in L_Q(v)$, then $u \prec v \preceq a$ for some $a \in A$, so $u \in \downarrow A$, and $u \notin A$ since A is an antichain; thus D is a down-set. For $a \in A$ we have $L_Q(a) \subseteq D$ by the same argument, so the edges into A from below leave D, and Lemma 5.3 gives $\sum_{a \in A} \mathrm{In}(a) \le I(D)$. By Lemma 2.1(i), $\lambda_a b_a = p_a b_a + |L_Q(a)| b_a \le p_a b_a + \mathrm{In}(a)$. Summing over A proves the first inequality. For the second, $\lambda_a h_a \ge \lambda_a + h_a - 1 = d_a - 1$ when $\lambda_a, h_a \ge 1$, so $\lambda_a b_a \ge 1/d_a$. □

Corollary 5.5 (A single supported vertex)**.** *Suppose $G[Q]$ is connected, every edge of $G[Q]$ is supportive, one vertex $q \in Q$ has $p_q = 1$, all other vertices of Q have $p_v = 0$, and every vertex of $Q \setminus \{q\}$ has a neighbour of smaller degree in Q. Then $q \prec v$, and hence $d_q < d_v$, for all $v \in Q \setminus \{q\}$; every vertex of $Q \setminus \{q\}$ has degree larger than $d_q(d_q - 1)$; and every antichain $A \subseteq Q \setminus \{q\}$ satisfies $\sum_{a \in A,\, h_a \ge 1} 1/d_a \le 1/d_q$.*

Proof. Following lower neighbours from any $v \neq q$ ends at a vertex without lower neighbour in Q, which must be q; so $q \prec v$. Hence all neighbours of q in Q are higher, $\lambda_q = 1$, $h_q = d_q - 1$ and $I(Q) = b_q = 1/d_q$. A vertex $w \neq q$ of minimum degree in $Q \setminus \{q\}$ can only have q as a lower neighbour, so $\lambda_w = 1$, $b_w = 1/d_w$, and supportiveness of qw means $\frac{1}{d_q(d_q-1)} = a_q > b_w = \frac{1}{d_w}$. The last claim is Theorem 5.4. □

Let us call a *configuration* a connected graph Q together with $U \subseteq V(Q)$; put $p_v = 1$ for $v \in U$ and $p_v = 0$ otherwise, $d_v = d_Q(v) + p_v$, and call it *valid* if $d_v \ge 2$ for all v, every vertex outside U has a neighbour of smaller degree in Q, and every edge of Q is supportive when each external neighbour is regarded as a lower neighbour. By Theorem 3.2, the core K of a minimal counterexample to (A), with U the set of vertices carrying a pendant path, is a valid configuration.

Proposition 5.6 (Computer-assisted)**.** *For $3 \le |V(Q)| \le 11$ there is no valid configuration with $|U| \le 2$. For $|V(Q)| = 2$ the only valid configuration with $|U| \le 2$ is $Q = K_2$ with $U = V(Q)$, i.e. the path P_4.*

Method. The program `lemu.c` (Appendix A) tests every connected graph Q on $n \le 11$ vertices produced by `geng -c` and every U with $|U| \in \{1, 2\}$, using the exact test of Lemma 2.1. □

Hence a minimal counterexample to (A) whose core has at most 11 vertices has at least three pendant paths, and, by Table 1, every bag with at most 11 core vertices has at least three pendant vertices.

6. Exhaustive verification through eleven vertices

Theorem 6.1 (Computer-assisted)**.** *Let G be a connected graph with $2 \le |V(G)| \le 11$. Then:*

(a) $R'(G) \ge \operatorname{rad}(G) - \frac{1}{2}$; whenever equality holds, $\operatorname{diam}(G) = 2\operatorname{rad}(G) - 1$;
(b) if G has at most one vertex of degree one, then $R'(G) \ge \operatorname{rad}(G)$;
(c) if G is not a path on an even number $n \ge 4$ of vertices, then $R(G) \ge \operatorname{rad}(G)$, with equality if and only if $G \in \{K_2, C_4, C_6, C_8, C_{10}\}$; otherwise $R(G) - \operatorname{rad}(G) \ge 0.2700556$.

Method. The program `verify.c` (Appendix A) reads the output of `geng -c` n for $n \le 11$. The radius is computed by bit-parallel breadth-first search. For $n \le 11$ every degree is at most 10, so $2520\,R'(G)$ is an integer ($2520 = \operatorname{lcm}(1, \dots, 10)$), and (a) and (b) are decided in exact integer arithmetic. For (c), $R(G)$ is a sum of at most 55 terms $1/\sqrt{d_u d_v}$, each computed with correctly rounded square root and division, so the computed sum has absolute error below 10^{-12}; all graphs with $|R(G) - \operatorname{rad}(G)| < 10^{-9}$ are printed and turn out to be exactly $K_2, C_4, C_6, C_8, C_{10}$, for which $R = \operatorname{rad}$ holds exactly. The diameter statement in (a) was checked on the list of all equality graphs. The case $n = 11$ was split into two halves with `geng`'s `res/mod` option; each half took about six and a half minutes on one core. The counts of graphs agree with the numbers of connected graphs (OEIS A001349 [11]). Table 2 summarizes the run. An independent Python script (Appendix A) with exact rational arithmetic and `networkx` reproduces all counts of Table 2 and the equality graphs in (c) for $n \le 8$. □

n	connected graphs	eq. in (A)	≤ 1 leaf	eq. in $R' \ge \operatorname{rad}$	min $R - \operatorname{rad}$
2	1	0	0	0	–
3	2	0	1	0	0.4142136
4	6	1	4	1	0.7320508
5	21	1	16	4	0.2700556
6	112	2	90	3	0.5606602
7	853	4	731	7	0.2700556
8	11 117	9	10 108	11	0.5606602
9	261 080	17	248 551	19	0.2700556
10	11 716 571	38	11 461 640	33	0.5606602
11	1 006 700 565	75	997 820 259	52	0.2700556

Table 2. Exhaustive verification. Columns: number of connected graphs; number with equality in (A); number with at most one vertex of degree one; number of those with $R' = \operatorname{rad}$; minimum of $R(G) - \operatorname{rad}(G)$ over graphs other than even paths and the equality graphs $K_2, C_4, \dots, C_{10}$. No violation of (A), of $R' \ge \operatorname{rad}$ for graphs with at most one leaf, or of Conjecture 1.2 occurs. For odd $n \ge 5$ it is attained by a path with two leaves attached at one end, where $R - \operatorname{rad} = \frac{2}{\sqrt{3}} + \frac{1}{\sqrt{6}} + \frac{1}{\sqrt{2}} - 2$.

The equality graphs of (A) include the even paths, the stars with one subdivided edge, and for $n \ge 7$ also graphs with cycles; all of them are extremal for Theorem 2.5 with odd diameter. The graphs with at most one leaf and $R'(G) = \operatorname{rad}(G)$ include the even cycles and the complete bipartite graphs $K_{2,m}$.

7. Conjectures and the remaining gap

The first conjecture is the general form of [4, Proposition 4.3]. By Corollary 3.5 it implies (A) and therefore Conjecture 1.1.

Conjecture 7.1 (Bag inequality)**.** *Every bag H satisfies $R'(H) \ge \operatorname{rad}(C(H)) + 1$.*

Evidence: Theorem 4.1 and the proof of [4] for maximum degree four. The difficulty is visible in Table 1. One would like to combine a bound $R'(C) \geq \mathrm{rad}(C)$ for the bridgeless core (which holds for all bridgeless graphs on at most 11 vertices by Theorem 6.1(b)) with a bound $R'(H) - R'(C) \geq 1$ for the drop (1). The drop, however, can be as small as $\frac{3}{4}$, and the slack in (B) vanishes for $K_{2,3}$; so the two parts cannot be separated, and the core and the pendant set must be treated together. Theorem 5.1 expresses the drop through the slacks of the strict core edges, which is the form in which the supportive condition enters. Table 3 suggests that the slack in (B) grows with the order of the core: for each value of the radius, the minimum of $R'(H)$ increases with $|V(C)|$, roughly by $\frac{1}{3}$ per vertex. For radius two and core order $5 \leq |V(C)| \leq 8$ the minimum is attained by $K_{2,m}$ with a pendant vertex at each of its m vertices of degree two (for $m = 3$ also at the other two vertices; for $m \geq 4$ pendant vertices there do not change the value), where $R'(H) = 2 + \frac{m}{3}$.

$\lvert V(C)\rvert$	$\mathrm{rad}(C) = 1$	$\mathrm{rad}(C) = 2$	$\mathrm{rad}(C) = 3$	$\mathrm{rad}(C) = 4$
4	29/12	–	–	–
5	11/4	3	–	–
6	47/15	10/3	–	–
7	7/2	11/3	–	–
8	107/28	4	14/3	–
9	33/8	361/84	5	–
10	53/12	55/12	16/3	6

TABLE 3. Minimum of $R'(H)$ over all bags H with given core order and core radius (exact values). The bag inequality (B) asks for at least $\mathrm{rad}(C)+1$, that is $2, 3, 4, 5$ in the four columns. Computed by a variant of `bags2.c` that records the minimum of $R'(H)$ separately for each value of $\mathrm{rad}(C)$.

Conjecture 7.2 (One leaf)**.** *Every connected graph with at most one vertex of degree one satisfies* $R'(G) \geq \mathrm{rad}(G)$.

Since $R \geq R'$, Conjecture 7.2 would give Conjecture 1.2 for all graphs with at most one leaf, in particular for all graphs of minimum degree at least two. It holds for $n \leq 11$ (Theorem 6.1(b)), with equality for the even cycles and the graphs $K_{2,m}$ among others. A randomized local search (simulated annealing over edge toggles, program `sa_oneleaf.c` in Appendix A) on graphs with 13 to 30 vertices found no violation of Conjecture 7.2 or of (A); this is weak evidence, since such searches tend to stay near the many equality graphs.

Remark 7.3 (Few supports)**.** Is there a valid configuration with $|V(Q)| \geq 3$ and $|U| \leq 2$? None exists with $|V(Q)| \leq 11$ (Proposition 5.6), but for the case relevant to bags this is no evidence at all. Consider a valid configuration with $U = \{q\}$ and $d_q \geq 3$ (in a bag or in the core of a minimal counterexample, $d_q \geq 3$). By Corollary 5.5 every other vertex has degree at least $m > d_q(d_q - 1)$. Let L_1 be the set of vertices whose only lower neighbour is q; then $L_1 \subseteq N(q)$, so $|L_1| \leq d_q - 1$, and $Q \neq \{q\} \cup L_1$ because degrees exceed $|L_1|$. A vertex w of minimum degree k in $Q \setminus (\{q\} \cup L_1)$ has all its lower neighbours in $\{q\} \cup L_1$, at least one of them, u, in L_1; hence $\lambda_w \leq d_q$ and supportiveness of uw gives $\frac{k-d_q}{k(k-1)} \leq b_w < a_u = \frac{1}{d_u(d_u-1)} \leq \frac{1}{m(m-1)}$. For $d_q \geq 3$ (so $m \geq 7$) this forces $k \geq 40$, hence $|V(Q)| \geq 41$. A negative answer to the question would imply that every bag has at least three pendant vertices and that a minimal counterexample to (A) has at least three pendant paths.

Summary of the gap. A proof of (A), and hence of Conjecture 1.1, along the present lines requires Conjecture 7.1 for cores of every order, whereas we can certify it only for cores of order

at most 11. The minimal counterexample analysis (Theorem 3.2) and the conservation identity (Theorem 5.1) reduce the problem to this single family of inequalities but do not prove them.

Appendix A. Programs

All computations used `geng` from nauty 2.8.8 [10], compiled C (`gcc -O3`), and Python 3 with `networkx` for the cross-checks. The listings below are complete. Typical invocations:

```
geng -cq 11 0/2 | ./verify          # half of the 11-vertex connected graphs
geng -cq -d2 10 | ./bags2           # all bags with 10-vertex cores
geng -cq -d2 11 3/8 | ./bags2       # one eighth of the 11-vertex cores
geng -cq 11 | ./lemu                # configurations with |U| <= 2
```

A.1. `verify.c` (Theorem 6.1)

```
/* verify.c -- exhaustive check of radius inequalities for the Randic index R and the
 * modified Randic index R'(G) = sum_{uv in E} 1/max(d(u),d(v)).
 *
 *   (A)  R'(G) >= rad(G) - 1/2                for every connected graph G     [implies Fajtlowicz]
 *   (A1) R'(G) >= rad(G)  if G has at most one vertex of degree 1
 *   (CH) R(G)  >= rad(G)  unless G is a path on an even number >= 4 of vertices [Caporossi-Hansen]
 *
 * Input: graph6 strings, one per line (e.g. nauty: geng -cq n | ./verify).
 * (A),(A1): exact integer arithmetic.  For n <= 11 every degree is <= 10 and 2520 = lcm(1..10),
 *           so 2520*R'(G) is an integer.  Graphs with a vertex of degree > 10 are rejected.
 * (CH):     R is summed in IEEE double precision.  Every term 1/sqrt(d_u d_v) is computed with
 *           correctly rounded sqrt and division, and at most 55 terms are added, so the absolute
 *           error of the computed sum is far below 1e-12.  Graphs with |R - rad| < 1e-9 are
 *           printed (tag NEAR) and decided separately in exact arithmetic.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

#define MAXN 16
#define SCALE 2520

static int n, deg[MAXN];
static unsigned nb[MAXN];
static double isq[MAXN][MAXN];

static void parse(const char *s) {
    int p = 0;
    n = s[p++] - 63;
    if (n < 1 || n > MAXN) { fprintf(stderr, "bad order\n"); exit(1); }
    memset(deg, 0, sizeof deg); memset(nb, 0, sizeof nb);
    int bits = 0, val = 0;
    for (int j = 1; j < n; j++)
        for (int i = 0; i < j; i++) {
            if (bits == 0) { val = s[p++] - 63; bits = 6; }
            bits--;
            if ((val >> bits) & 1) { nb[i] |= 1u << j; nb[j] |= 1u << i; deg[i]++; deg[j]++; }
        }
}

/* radius by bit-parallel BFS; returns -1 if the graph is disconnected */
static int radius(void) {
    unsigned all = (n == 32) ? 0xffffffffu : ((1u << n) - 1);
    int best = 1 << 30;
    for (int s = 0; s < n; s++) {
        unsigned seen = 1u << s, fr = 1u << s; int e = 0;
        while (seen != all) {
```

```
            unsigned nx = 0;
            for (unsigned f = fr; f; f &= f - 1) nx |= nb[__builtin_ctz(f)];
            nx &= ~seen;
            if (!nx) return -1;
            seen |= nx; fr = nx; e++;
            if (e >= best) break;          /* cannot improve the minimum */
        }
        if (e < best) best = e;
    }
    return best;
}

int main(void) {
    char line[256];
    for (int a = 1; a < MAXN; a++) for (int b = 1; b < MAXN; b++) isq[a][b] = 1.0 / sqrt((double)(a * b));
    long long cnt = 0, eqA = 0, violA = 0, cntA1 = 0, eqA1 = 0, violA1 = 0, violCH = 0, nearCH = 0,
     evenpaths = 0;
    long long minA = 1LL << 60, minA1 = 1LL << 60;
    double minCH = 1e9;
    char wCH[256] = "";
    while (fgets(line, sizeof line, stdin)) {
        int len = (int)strlen(line);
        while (len && (line[len - 1] == '\n' || line[len - 1] == '\r')) line[--len] = 0;
        parse(line);
        if (n < 2) continue;
        int r = radius();
        if (r < 0) { fprintf(stderr, "disconnected input %s\n", line); return 1; }
        long long S = 0; int leaves = 0, deg2 = 0, m = 0; double R = 0.0;
        for (int i = 0; i < n; i++) {
            if (deg[i] > 10) { fprintf(stderr, "degree > 10 not supported\n"); return 1; }
            if (deg[i] == 1) leaves++;
            if (deg[i] == 2) deg2++;
            for (unsigned f = nb[i] & ~((2u << i) - 1); f; f &= f - 1) {   /* neighbours j > i */
                int j = __builtin_ctz(f);
                int mx = deg[i] > deg[j] ? deg[i] : deg[j];
                S += SCALE / mx; R += isq[deg[i]][deg[j]]; m++;
            }
        }
        cnt++;
        /* (A) */
        long long sA = S - (long long)SCALE * r + SCALE / 2;
        if (sA < minA) minA = sA;
        if (sA < 0) { violA++; printf("VIOLATION(A) %s\n", line); } else if (sA == 0) eqA++;
        /* (A1) */
        if (leaves <= 1) {
            cntA1++;
            long long s1 = S - (long long)SCALE * r;
            if (s1 < minA1) minA1 = s1;
            if (s1 < 0) { violA1++; printf("VIOLATION(A1) %s\n", line); } else if (s1 == 0) eqA1++;
        }
        /* (CH) */
        int evenpath = (n >= 4 && n % 2 == 0 && m == n - 1 && leaves == 2 && deg2 == n - 2);
        if (evenpath) { evenpaths++; continue; }
        double sc = R - r;
        if (sc < minCH) { minCH = sc; strcpy(wCH, line); }
        if (fabs(sc) < 1e-9) { nearCH++; printf("NEAR(CH) %s r=%d R=%.15f\n", line, r, R); }
        else if (sc < 0) { violCH++; printf("VIOLATION(CH) %s r=%d R=%.15f\n", line, r, R); }
    }
    printf("SUMMARY graphs %lld | (A) min 2520*(R'-rad+1/2) = %lld, equality %lld, violations %lld"
           " | (A1) graphs with <=1 leaf %lld, min 2520*(R'-rad) = %lld, equality %lld, violations %lld"
           " | (CH) even paths skipped %lld, min R-rad = %.9f at %s, near-equality %lld, violations %lld\n",
           cnt, minA, eqA, violA, cntA1, cntA1 ? minA1 : 0, eqA1, violA1, evenpaths, minCH, wCH, nearCH,
     violCH);
    return 0;
```

```
}
```

A.2. bags2.c (Theorem 4.1)

```
/* bags2.c -- exhaustive check of the bag inequality R'(H) >= rad(C(H)) + 1 for every bag H whose core C(H)
 * has a given number n <= 11 of vertices.  Input: graph6 strings of all connected graphs with minimum degree
 * >= 2 (geng -c -d2 n); graphs with a bridge are skipped.  A bag is a core C plus a set U of core vertices,
 * each carrying one pendant vertex, such that every core edge is supportive (deleting it increases R').
 *
 * Forced pendants.  If a core vertex v has no core neighbour of smaller core-degree, then v must carry a
 * pendant in every bag: otherwise d_H(v) = cdeg(v) <= cdeg(w) <= d_H(w) for every neighbour w, so v is
 * locally minimal in H and its core edges are not supportive.  The same holds when cdeg(v) <= 3 (a vertex
 * of core-degree 2 without pendant is locally minimal; then a vertex of core-degree 3 without pendant has
 * only neighbours of H-degree >= 3).  Inside the enumeration we additionally require (before the exact
 * supportive test) that every core vertex outside U has a neighbour of smaller H-degree; this is implied
 * by the supportive test and only speeds it up.
 * Exact arithmetic: R'(H) is an integer multiple of 1/27720 (27720 = lcm(1,...,11)) for cores of order <= 11.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXN 16
#define SCALE 27720
static int n, cdeg[MAXN], d[MAXN], pend[MAXN];
static unsigned nb[MAXN];
static void parse(const char *s) {
    int p = 0; n = s[p++] - 63;
    if (n < 1 || n > 11) { fprintf(stderr, "order must be <= 11\n"); exit(1); }
    memset(cdeg, 0, sizeof cdeg); memset(nb, 0, sizeof nb);
    int bits = 0, val = 0;
    for (int j = 1; j < n; j++) for (int i = 0; i < j; i++) {
        if (bits == 0) { val = s[p++] - 63; bits = 6; }
        bits--;
        if ((val >> bits) & 1) { nb[i] |= 1u << j; nb[j] |= 1u << i; cdeg[i]++; cdeg[j]++; }
    }
}
static int connected_without(int a, int b) {
    unsigned all = (1u << n) - 1, seen = 1u, fr = 1u;
    while (fr) {
        unsigned nx = 0;
        for (unsigned f = fr; f; f &= f - 1) {
            int u = __builtin_ctz(f); unsigned m = nb[u];
            if (u == a) m &= ~(1u << b);
            if (u == b) m &= ~(1u << a);
            nx |= m;
        }
        nx &= ~seen; seen |= nx; fr = nx;
    }
    return seen == all;
}
static int radius(void) {
    unsigned all = (1u << n) - 1; int best = 1 << 30;
    for (int s = 0; s < n; s++) {
        unsigned seen = 1u << s, fr = 1u << s; int e = 0;
        while (seen != all) {
            unsigned nx = 0;
            for (unsigned f = fr; f; f &= f - 1) nx |= nb[__builtin_ctz(f)];
            nx &= ~seen; seen |= nx; fr = nx; e++;
        }
        if (e < best) best = e;
    }
    return best;
}
static int supportive(int i, int j) {
```

```
    int di = d[i], dj = d[j], bi = pend[i], bj = pend[j];
    for (unsigned f = nb[i]; f; f &= f - 1) { int w = __builtin_ctz(f); if (w != j && d[w] < di) bi++; }
    for (unsigned f = nb[j]; f; f &= f - 1) { int w = __builtin_ctz(f); if (w != i && d[w] < dj) bj++; }
    long long mx = di > dj ? di : dj;
    long long A = (long long)bi * dj * (dj - 1) * mx + (long long)bj * di * (di - 1) * mx;
    long long B = (long long)di * (di - 1) * dj * (dj - 1);
    return A > B;
}
int main(void) {
    char line[256];
    long long cores = 0, bags = 0, eq = 0, viol = 0, minpend = 1 << 30;
    long long minslack = 1LL << 60, mindrop = 1LL << 60;
    char wslack[256] = "", wdrop[256] = ""; unsigned wsU = 0, wdU = 0;
    while (fgets(line, sizeof line, stdin)) {
        int len = (int)strlen(line);
        while (len && (line[len - 1] == '\n' || line[len - 1] == '\r')) line[--len] = 0;
        parse(line);
        unsigned forced = 0; int opt[MAXN], no = 0;
        for (int i = 0; i < n; i++) {
            int lowcore = 0;
            for (unsigned f = nb[i]; f; f &= f - 1) if (cdeg[__builtin_ctz(f)] < cdeg[i]) { lowcore = 1;
    break; }
            if (cdeg[i] <= 3 || !lowcore) forced |= 1u << i; else opt[no++] = i;
        }
        int ok = 1;
        for (int i = 0; i < n && ok; i++)
            for (unsigned f = nb[i] & ~((2u << i) - 1); f && ok; f &= f - 1)
                if (!connected_without(i, __builtin_ctz(f))) ok = 0;
        if (!ok) continue;
        cores++;
        int rc = -1; long long Mcore = 0;
        for (unsigned mask = 0; mask < (1u << no); mask++) {
            unsigned U = forced;
            for (int k = 0; k < no; k++) if ((mask >> k) & 1) U |= 1u << opt[k];
            for (int i = 0; i < n; i++) { pend[i] = (U >> i) & 1; d[i] = cdeg[i] + pend[i]; }
            int good = 1;
            for (int i = 0; i < n && good; i++) {           /* quick necessary condition */
                if (pend[i]) continue;
                int has = 0;
                for (unsigned f = nb[i]; f; f &= f - 1) if (d[__builtin_ctz(f)] < d[i]) { has = 1; break; }
                if (!has) good = 0;
            }
            for (int i = 0; i < n && good; i++)
                for (unsigned f = nb[i] & ~((2u << i) - 1); f && good; f &= f - 1)
                    if (!supportive(i, __builtin_ctz(f))) good = 0;
            if (!good) continue;
            if (rc < 0) {
                rc = radius();
                for (int i = 0; i < n; i++)
                    for (unsigned f = nb[i] & ~((2u << i) - 1); f; f &= f - 1) {
                        int j = __builtin_ctz(f); int mx = cdeg[i] > cdeg[j] ? cdeg[i] : cdeg[j]; Mcore +=
    SCALE / mx;
                    }
            }
            bags++;
            if (__builtin_popcount(U) < minpend) minpend = __builtin_popcount(U);
            long long M = 0;
            for (int i = 0; i < n; i++) {
                if (pend[i]) M += SCALE / d[i];
                for (unsigned f = nb[i] & ~((2u << i) - 1); f; f &= f - 1) {
                    int j = __builtin_ctz(f); int mx = d[i] > d[j] ? d[i] : d[j]; M += SCALE / mx;
                }
            }
            long long slack = M - (long long)SCALE * (rc + 1);
```

```
            if (slack < minslack) { minslack = slack; strcpy(wslack, line); wsU = U; }
            if (slack < 0) { viol++; printf("VIOLATION %s U=%x\n", line, U); }
            else if (slack == 0) { eq++; printf("EQUALITY %s U=%x\n", line, U); }
            long long drop = M - Mcore;
            if (drop < mindrop) { mindrop = drop; strcpy(wdrop, line); wdU = U; }
        }
    }
    printf("SUMMARY cores %lld bags %lld | min %d*(R'(H)-rad(C)-1) = %lld at %s U=%x | equality %lld
     violations %lld"
           " | min %d*(R'(H)-R'(C)) = %lld at %s U=%x | min #pendants %lld\n",
           cores, bags, SCALE, bags ? minslack : 0, wslack, wsU, eq, viol, SCALE, bags ? mindrop : 0, wdrop,
      wdU,
           bags ? minpend : 0);
    return 0;
}
```

A.3. `bags_xcheck.py` (independent cross-check of Theorem 4.1 for core order ≤ 7)

```
"""Independent brute-force cross-check of bags.c for small cores (no shortcuts, no edge-deletion formula).
For every bridgeless connected graph C on n vertices and EVERY subset U of V(C), build H = C + one pendant
vertex at each u in U, test the bag conditions directly (recomputing R' after each core-edge deletion),
and record R'(H) - rad(C) - 1 and R'(H) - R'(C) as exact fractions."""
import sys, itertools, subprocess
from fractions import Fraction as F
import networkx as nx

def Rp(G):
    return sum(F(1, max(G.degree(u), G.degree(v))) for u, v in G.edges())

def main(n, geng):
    out = subprocess.run([geng, '-cq', '-d2', str(n)], capture_output=True, text=True).stdout.split()
    cores = bags = 0; minslack = None; mindrop = None; eq = []
    for s in out:
        C = nx.from_graph6_bytes(s.encode())
        if nx.has_bridges(C): continue
        cores += 1
        rc = nx.radius(C); MC = Rp(C)
        V = list(C.nodes())
        for k in range(len(V) + 1):
            for U in itertools.combinations(V, k):
                H = C.copy()
                for u in U: H.add_edge(u, ('p', u))
                MH = Rp(H)
                ok = True
                for e in C.edges():
                    H2 = H.copy(); H2.remove_edge(*e)
                    if not (Rp(H2) > MH): ok = False; break
                if not ok: continue
                bags += 1
                slack = MH - rc - 1; drop = MH - MC
                if minslack is None or slack < minslack: minslack = slack
                if mindrop is None or drop < mindrop: mindrop = drop
                if slack == 0: eq.append((s, U))
    print(f"n_C={n}: cores {cores} bags {bags} min slack {minslack} min drop {mindrop} equality {eq}")

if __name__ == '__main__':
    geng = sys.argv[1]
    for n in range(3, int(sys.argv[2]) + 1):
        main(n, geng)
```

A.4. `lemu.c` (Proposition 5.6)

```
/* lemu.c -- search for "valid configurations" with few externally supported vertices.
 * Input: connected graphs Q (graph6).  For every set U of size 1 or 2 (p_v = 1 for v in U, else 0):
```

```
 *   d_v = deg_Q(v) + p_v ;  require d_v >= 2 for all v ;
 *   every v not in U has a Q-neighbour w with d_w < d_v ;
 *   every Q-edge uv is supportive:  -1/max(d_u,d_v) + b_u/(d_u(d_u-1)) + b_v/(d_v(d_v-1)) > 0,
 *   where b_x = p_x + #{w ~ x in Q, w != other end, d_w < d_x}.
 * Prints every valid configuration found (tag VALID) and counts by |U|.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXN 20
static int n, cdeg[MAXN], d[MAXN], pend[MAXN];
static unsigned nb[MAXN];
static void parse(const char *s) {
    int p = 0; n = s[p++] - 63;
    memset(cdeg, 0, sizeof cdeg); memset(nb, 0, sizeof nb);
    int bits = 0, val = 0;
    for (int j = 1; j < n; j++) for (int i = 0; i < j; i++) {
        if (bits == 0) { val = s[p++] - 63; bits = 6; }
        bits--;
        if ((val >> bits) & 1) { nb[i] |= 1u << j; nb[j] |= 1u << i; cdeg[i]++; cdeg[j]++; }
    }
}
static int supportive(int i, int j) {
    int di = d[i], dj = d[j], bi = pend[i], bj = pend[j];
    for (unsigned f = nb[i]; f; f &= f - 1) { int w = __builtin_ctz(f); if (w != j && d[w] < di) bi++; }
    for (unsigned f = nb[j]; f; f &= f - 1) { int w = __builtin_ctz(f); if (w != i && d[w] < dj) bj++; }
    long long mx = di > dj ? di : dj;
    long long A = (long long)bi * dj * (dj - 1) * mx + (long long)bj * di * (di - 1) * mx;
    long long B = (long long)di * (di - 1) * dj * (dj - 1);
    return A > B;
}
static int valid(unsigned U) {
    for (int i = 0; i < n; i++) { pend[i] = (U >> i) & 1; d[i] = cdeg[i] + pend[i]; if (d[i] < 2) return 0;
     }
    for (int i = 0; i < n; i++) {
        if (pend[i]) continue;
        int has = 0;
        for (unsigned f = nb[i]; f; f &= f - 1) if (d[__builtin_ctz(f)] < d[i]) { has = 1; break; }
        if (!has) return 0;
    }
    for (int i = 0; i < n; i++)
        for (unsigned f = nb[i] & ~((2u << i) - 1); f; f &= f - 1)
            if (!supportive(i, __builtin_ctz(f))) return 0;
    return 1;
}
int main(void) {
    char line[256]; long long graphs = 0, c1 = 0, c2 = 0;
    while (fgets(line, sizeof line, stdin)) {
        int len = (int)strlen(line);
        while (len && (line[len - 1] == '\n' || line[len - 1] == '\r')) line[--len] = 0;
        parse(line); graphs++;
        /* quick filter: the vertices outside U need lower neighbours, so the minimum of
           deg_Q over V \ U exceeds min over U of deg_Q + 1 ... we simply test all U */
        for (int a = 0; a < n; a++) {
            if (valid(1u << a)) { c1++; printf("VALID1 %s U={%d}\n", line, a); }
            for (int b = a + 1; b < n; b++)
                if (valid((1u << a) | (1u << b))) { c2++; printf("VALID2 %s U={%d,%d}\n", line, a, b); }
        }
    }
    printf("SUMMARY graphs %lld valid |U|=1: %lld  |U|=2: %lld\n", graphs, c1, c2);
    return 0;
}
```

A.5. `verify_xcheck.py` (independent cross-check of Theorem 6.1 for $n \le 8$)

```
"""Independent check of Theorem 6.1 (a),(b) counts for small n with exact rationals (networkx)."""
import sys, subprocess, math
from fractions import Fraction as F
import networkx as nx
geng = sys.argv[1]
for n in range(2, int(sys.argv[2]) + 1):
    out = subprocess.run([geng, '-cq', str(n)], capture_output=True, text=True).stdout.split()
    cnt = eqA = le1 = eqA1 = 0; viol = 0; cheq = []
    for s in out:
        G = nx.from_graph6_bytes(s.encode()); cnt += 1
        r = nx.radius(G)
        Rp = sum(F(1, max(G.degree(u), G.degree(v))) for u, v in G.edges())
        if Rp < r - F(1, 2): viol += 1
        if Rp == r - F(1, 2): eqA += 1
        leaves = sum(1 for _, d in G.degree() if d == 1)
        if leaves <= 1:
            le1 += 1
            if Rp < r: viol += 1
            if Rp == r: eqA1 += 1
        R = sum(1 / math.sqrt(G.degree(u) * G.degree(v)) for u, v in G.edges())
        evenpath = n >= 4 and n % 2 == 0 and nx.is_tree(G) and max(d for _, d in G.degree()) == 2
        if not evenpath:
            if R < r - 1e-9: viol += 1
            if abs(R - r) < 1e-9: cheq.append(s)
    print(n, cnt, eqA, le1, eqA1, 'viol', viol, 'CH-eq', cheq)
```

A.6. `sa_oneleaf.c` (randomized search mentioned in Section 7)

```
/* sa_oneleaf.c -- randomized local search for graphs with at most one leaf minimizing R'(G) - rad(G)
 * (Conjecture "one leaf": R'(G) >= rad(G)) and, in mode 1, for all connected graphs minimizing
 * R'(G) - rad(G) + 1/2 (inequality (A)).  Moves: toggle a random pair; reject disconnected graphs and,
 * in mode 0, graphs with two or more leaves.  Simulated annealing on the slack (double precision is enough
 * for a search; any hit would be re-checked exactly). Usage: sa_oneleaf mode n iters seed */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <string.h>
#define MAXN 64
static int n, adj[MAXN][MAXN], deg[MAXN];
static int connected_rad(int *rad) {
    int best = 1 << 30, dist[MAXN], q[MAXN];
    for (int s = 0; s < n; s++) {
        for (int i = 0; i < n; i++) dist[i] = -1;
        int h = 0, t = 0, e = 0; q[t++] = s; dist[s] = 0;
        while (h < t) { int u = q[h++]; if (dist[u] > e) e = dist[u];
            for (int v = 0; v < n; v++) if (adj[u][v] && dist[v] < 0) { dist[v] = dist[u] + 1; q[t++] = v; }
     }
        if (t < n) return 0;
        if (e < best) best = e;
    }
    *rad = best; return 1;
}
static double slack(int mode, int *ok) {
    int leaves = 0, r;
    for (int i = 0; i < n; i++) { if (deg[i] == 0) { *ok = 0; return 0; } if (deg[i] == 1) leaves++; }
    if (mode == 0 && leaves > 1) { *ok = 0; return 0; }
    if (!connected_rad(&r)) { *ok = 0; return 0; }
    double M = 0;
    for (int i = 0; i < n; i++) for (int j = i + 1; j < n; j++) if (adj[i][j]) M += 1.0 / (deg[i] > deg[j] ?
      deg[i] : deg[j]);
    *ok = 1;
    return mode == 0 ? M - r : M - r + 0.5;
```

```
}
static void toggle(int i, int j) { if (adj[i][j]) { adj[i][j] = adj[j][i] = 0; deg[i]--; deg[j]--; } else {
     adj[i][j] = adj[j][i] = 1; deg[i]++; deg[j]++; } }
int main(int argc, char **argv) {
    int mode = atoi(argv[1]); n = atoi(argv[2]); long iters = atol(argv[3]); srand(atoi(argv[4]));
    memset(adj, 0, sizeof adj); memset(deg, 0, sizeof deg);
    for (int i = 0; i < n; i++) toggle(i, (i + 1) % n);           /* start from a cycle */
    int ok; double cur = slack(mode, &ok), best = cur; double T = 0.5;
    for (long it = 0; it < iters; it++) {
        int i = rand() % n, j = rand() % n; if (i == j) continue;
        toggle(i, j);
        double s = slack(mode, &ok);
        if (!ok) { toggle(i, j); continue; }
        if (s <= cur || exp((cur - s) / T) > (double)rand() / RAND_MAX) {
            cur = s;
            if (s < best) { best = s; }
            if (s < -1e-9) {
                printf("CANDIDATE slack=%.6f n=%d edges:", s, n);
                for (int a = 0; a < n; a++) for (int b = a + 1; b < n; b++) if (adj[a][b]) printf(" %d-%d",
    a, b);
                printf("\n"); fflush(stdout);
            }
        } else toggle(i, j);
        T *= 0.9999995; if (T < 0.01) T = 0.3;
    }
    printf("mode %d n %d best slack %.6f\n", mode, n, best);
    return 0;
}
```

References

[1] V. Andova, M. Knor, P. Potočnik and R. Škrekovski, On a variation of the Randić index, *Australas. J. Combin.* **56** (2013) 61–75.

[2] G. Caporossi and P. Hansen, Variable neighborhood search for extremal graphs. 1. The AutoGraphiX system, *Discrete Math.* **212** (2000) 29–44.

[3] Supportive edges, radius-stable peeling and rooted Yang–Lu replacement: the Randić–radius conjecture for graphs of cyclomatic number at most 8, manuscript [054] in the *Verifiable Mathematical Reasoning Corpus*, `https://github.com/QEEDDD/Verifiable-Mathematical-Reasoning-Corpus` (unrefereed; accessed October 2026).

[4] M. Cygan, M. Pilipczuk and R. Škrekovski, On the inequality between radius and Randić index for graphs, *MATCH Commun. Math. Comput. Chem.* **67** (2012) 451–466.

[5] H. Deng, Z. Tang and J. Zhang, On a conjecture of Randić index and graph radius, *Filomat* **29** (2015) 1369–1375, doi:10.2298/FIL1506369D; arXiv:1210.2543.

[6] M. I. Doig, Randić index, radius, and diameter for cactus graphs, arXiv:2107.00071 (2021).

[7] M. I. Doig, Block eccentricity, a radius bound, and an application to the Randić index, arXiv:2309.11613 (2023).

[8] Z. Dvořák, B. Lidický and R. Škrekovski, Randić index and the diameter of a graph, *European J. Combin.* **32** (2011) 434–442.

[9] S. Fajtlowicz, On conjectures of Graffiti, *Discrete Math.* **72** (1988) 113–118.

[10] B. D. McKay and A. Piperno, Practical graph isomorphism, II, *J. Symbolic Comput.* **60** (2014) 94–112.

[11] OEIS Foundation, Sequence A001349 (number of connected graphs with n nodes), *The On-Line Encyclopedia of Integer Sequences*, `https://oeis.org/A001349`.

[12] OEIS Foundation, Sequence A007146 (number of unlabeled simple connected bridgeless graphs with n nodes), *The On-Line Encyclopedia of Integer Sequences*, `https://oeis.org/A007146`.

[13] M. Randić, On characterization of molecular branching, *J. Amer. Chem. Soc.* **97** (1975) 6609–6615.

[14] Y. Yang and L. Lu, The Randić index and the diameter of graphs, *Discrete Math.* **311** (2011) 1333–1343; arXiv:1104.0426.

[15] Z. You and B. Liu, On a conjecture of the Randić index, *Discrete Appl. Math.* **157** (2009) 1766–1772.